

MEDIATEK

MT8676 Hypervisor OP-TEE User Manual

version: 1.0
Release date: 2024-12-20

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-12-20	Fei Yan	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	4
List of Tables	4
1 Overview	5
1.1 Trustzone Function Overview	5
1.2 OP-TEE Overview	7
2 OP-TEE Framework and Functionality	9
2.1 OP-TEE Core Components with Virtualization Enabled	9
2.2 Enabling OP-TEE on MediaTek Platform	11
2.2.1 Yocto Branch	12
2.2.2 Android Branch	13
2.3 Secure Storage Mechanism	14
2.3.1 REE File System	14
2.3.2 RPMB File System	15
2.4 Introduction to CA and TA	16
3 Application Development Based on OP-TEE	18
3.1 Writing CA and TA with GP API	18
3.2 Example Program	19
3.2.1 CA Files	19
3.2.2 TA Files	21
3.3 Compilation and Execution	22
3.3.1 Yocto Domain	23
3.3.2 Android Domain	26
4 OP-TEE Testing and Debugging	28
4.1 OP-TEE Self-Test Method	28
4.2 Introduction to OP-TEE Related Logs	29
4.2.1 User Space	29
4.2.2 Linux Kernel	30
4.2.3 Secure World	30
4.3 Common OP-TEE Exceptions and Analysis	31
5 Appendix	33
5.1 Reference Documents	33
Exhibit 1 Terms and Conditions	34

List of Figures

Figure 1-1. Normal World and Secure World5

Figure 1-2. Arm exception levels6

Figure 1-3. Reserverd OP-TEE memory6

Figure 1-4. OP-TEE architecture8

Figure 2-1. Composition of OP-TEE without virtualization10

Figure 2-2. Composition of OP-TEE with virtualization11

Figure 2-3. Secure storage based on OP-TEE14

Figure 4-1. OP-TEE log31

List of Tables

Table 4-1. Xtest test items28

1 Overview

This document introduces the features of OP-TEE, its structural design, and the composition of its various components. And the following topics are explained: 1. Writing CA and TA. 2. Performing self-testing. 3. Debugging.

The structure of this document is as follows:

- Chapter 1 Presents an overview that introduces the basic concepts of TrustZone and OP-TEE.
- Chapter 2 Presents the framework and functions of OP-TEE.
- Chapter 3 Describes the development of applications based on OP-TEE.
- Chapter 4 Describes testing and debugging, including debugging methods for the OP-TEE OS and Trusted Application (TA).
- Chapter 5 Lists the reference documents.

1.1 Trustzone Function Overview

TrustZone is the name of the security architecture in the Arm A-profile architecture. TrustZone provides two execution environments and enforces hardware-based isolation between them across the entire system, as illustrated in Figure 1-1.

Figure 1-1 shows schematic diagram of the Normal World and the Secure World. The Normal World runs a rich software stack, which typically includes a large number of applications, a complex operating system (such as Linux), and possibly a hypervisor. This software stack is large and complex. Although measures can be taken to enhance its security, it remains more susceptible to attacks due to its large attack surface.

In contrast, the Secure World runs a smaller and simpler software stack known as the Trusted Execution Environment (TEE). Typically, the TEE includes several trusted services hosted by a lightweight Kernel. These trusted services provide functions such as key management. The attack surface of this software stack is significantly smaller, which greatly reduces its vulnerability to attacks.

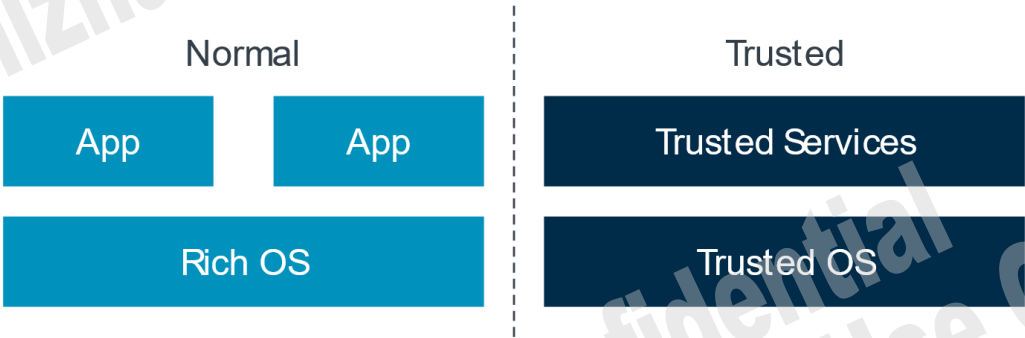


Figure 1-1. Normal World and Secure World

Armv8 and v9 divide the CPU into four exception levels, namely EL0, EL1, EL2, and EL3. At the EL0, EL1, and EL2 levels, the processor can be in either a secure state or a non-secure state, which is controlled by the SCR_EL3.NS bit. You will typically see the following notation:

- NS-EL1: Non-secure state, Exception Level 1;
- S-EL1: Secure state, Exception Level 1.

EL3 is always in a secure state, regardless of the value of the SCR_EL3.NS bit.

The arrangement of secure states and exception levels is shown in Figure 1-2.

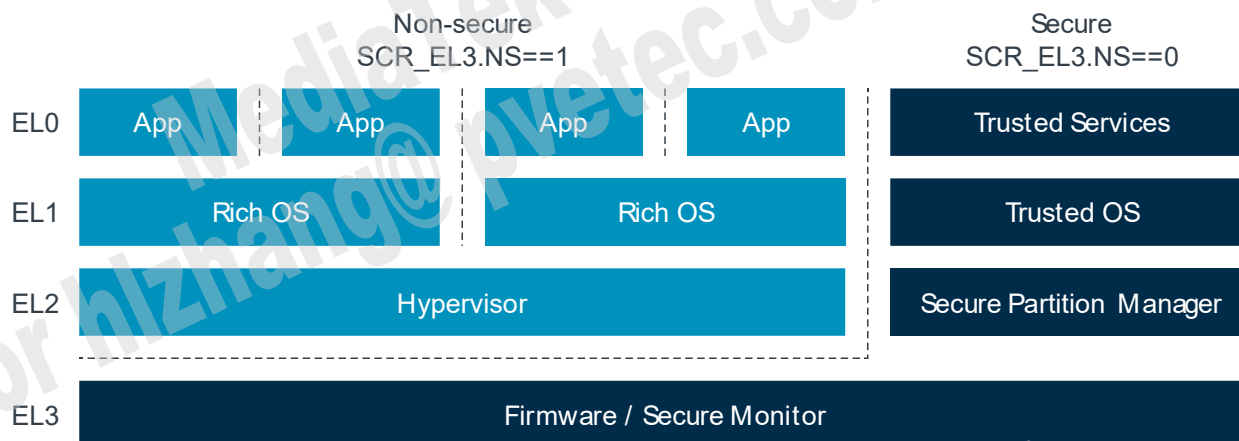


Figure 1-2. Arm exception levels

To ensure the security of TrustZone, a series of hardware isolation technologies is provided by Arm, including interrupt isolation, on-chip RAM/ROM isolation, off-chip RAM/ROM isolation, peripheral hardware isolation, and external RAM and ROM isolation. The provided isolation technologies can be referenced in Arm technical documentation.

A more powerful Memory Protection Unit is offered by MediaTek to ensure that only secure CPUs can access the memory areas where TrustZone resides. Non-secure CPUs and various peripherals are prevented from accessing these memory areas through multiple means.

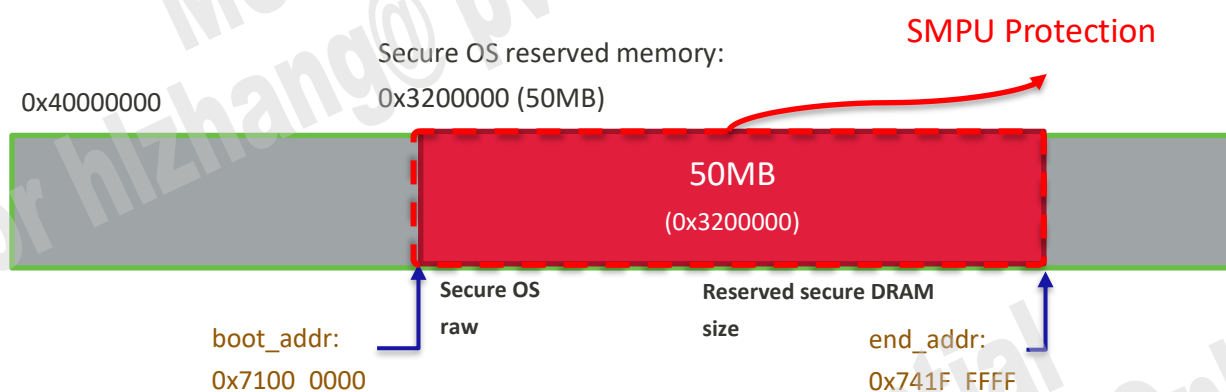


Figure 1-3. Reserved OP-TEE memory

Figure 1-3 is an example diagram of reserved TEE memory and SMPU protecting this reserved memory. This memory is only accessible by the Secure CPU, meaning that, as mentioned earlier, only the CPU in S-EL0, S-EL1, or EL3 can access it. Other scenarios, such as user space programs, Linux Kernel programs, and hypervisor programs, do not have the right to access this memory.

1.2 OP-TEE Overview

OP-TEE is an open-source Secure OS. Figure 1-4 is an architecture diagram provided by the official OP-TEE documentation. The OP-TEE core runs in S-EL1, and trusted applications run in S-EL0. Trusted applications communicate with the OP-TEE core through the TEE Internal API. The TEE Internal API is a standard API developed by the GlobalPlatform organization. GlobalPlatform is dedicated to developing standard APIs that support not only OP-TEE but also many different TEEs. Adopting the GP standard means that TEE applications designed for other TEEs supporting the GP interface can be easily ported to OP-TEE, and vice versa, TEE applications designed for OP-TEE can be easily ported to other TEEs supporting the GP interface.

In the non-secure state, there is an OP-TEE driver in the kernel space. It is responsible for handling low-level communication with the OP-TEE core. In the non-secure user space (EL0), there is a user space library (libopenteec.so) that implements the GlobalPlatform API. The TEE Client API is the interface used by applications to access TAs (Trusted Applications). OP-TEE also includes a component called tee-supplciant. The tee-supplciant handles transactions that need to be returned to the REE (Rich Execution Environment) world for processing, such as loading UTAs (User Trusted Applications) from the file system, or when the TEE secure storage function needs to save encrypted data in Flash or RPMB. Since the TEE itself cannot read or write Flash or RPMB, it needs to switch back to the REE to complete the read/write operations through the tee-supplciant.

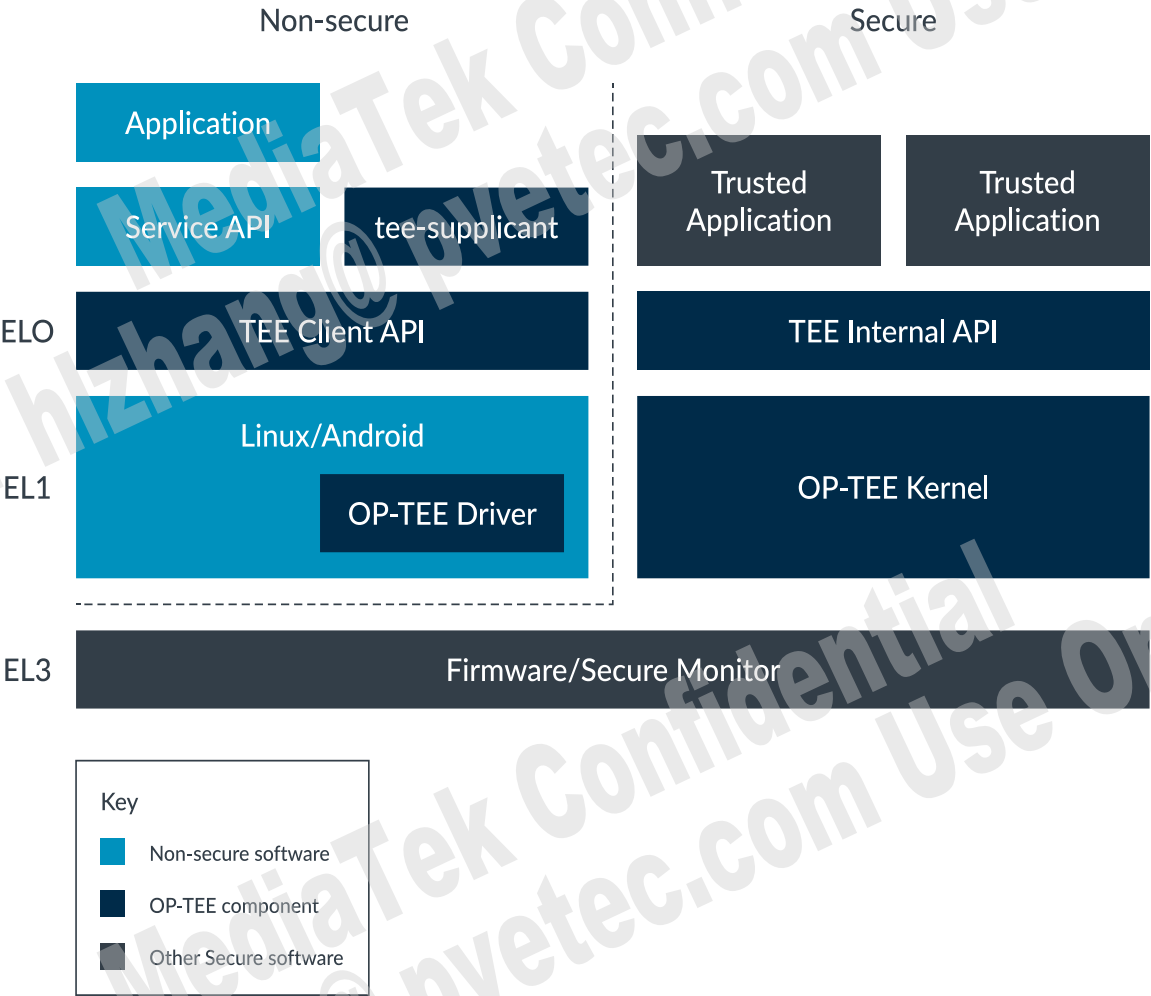


Figure 1-4. OP-TEE architecture

2 OP-TEE Framework and Functionality

2.1 OP-TEE Core Components with Virtualization Enabled

This chapter focuses on the composition of OP-TEE when virtualization is enabled.

Figure 2-1 shows the components of OP-TEE when virtualization is not enabled. The components are as follows:

- Tee-supplciant: The TEE daemon that handles tasks requiring a switch back to the REE, such as loading UTAs and secure storage read/write operations.
- libopentec.so: Provides the client-side GP API, used for communication between CA and TA.
- OPTEE linux driver: Provides APIs such as shared memory, executes SMC, and communicates with the Secure World.
- OPTEE OS: Provides all the functionalities at the OP-TEE kernel layer.
- PTA (Pseudo Trusted Applications): Provides APIs that can be called by CA or UTA. Some system services are provided as PTAs. PTAs run at the OP-TEE kernel layer and are packaged together with OP-TEE OS.

Applications based on OP-TEE:

Applications run on OP-TEE as CA/TA.

- CA (Client Agent): Runs in the REE (Non-Secure World).
- TA (Trusted App): Runs in the TEE (Secure World). Operations and keys that need protection can be placed in the TA.

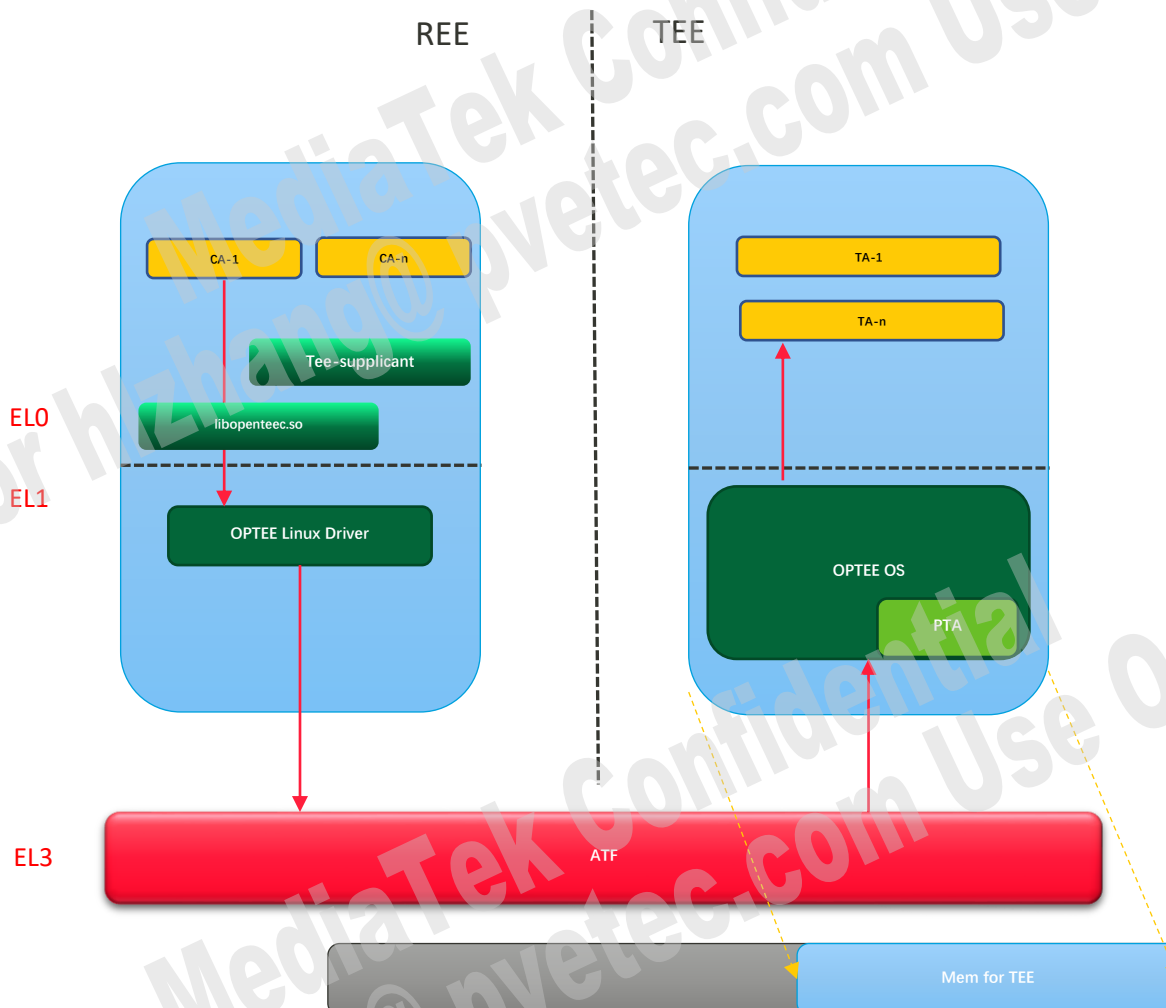


Figure 2-1. Composition of OP-TEE without virtualization

When virtualization is not enabled, the initialization of OP-TEE takes place upon the jump from ATF to BL32.

When OP-TEE virtualization is enabled, OP-TEE will create a corresponding OP-TEE VM for each Host VM within the TEE. As shown in Figure 2-2, the L+L+A architecture includes two Linux virtual machines and one Android virtual machine, namely:

- SOS (Linux Virtual Machine)
- UOS (Android Virtual Machine)
- UOS (Linux virtual machine, also known as the T-Box domain)

After enabling virtualization, when ATF jumps to BL32 for initialization, it will only initialize the OP-TEE nexus core layer. Before each virtual machine is created, the Hypervisor layer will send an SMC command (`OPTEE_SMC_VM_CREATED`) to OP-TEE to create the corresponding OP-TEE virtual machine and initialize the necessary data structures for the OP-TEE VM in the pre-allocated secure memory.

The TEE requests from the SOS (Linux virtual machine) will enter OP-TEE-VM1. When OP-TEE-VM1 returns to the REE through RPC or RET, it will also return to the SOS domain.

Requests from the UOS (Android Virtual Machine) enter OP-TEE-VM2. Upon returning from the TEE, responses return to UOS (A).

The same applies to the UOS (Linux Virtual Machine), also referred to as the T-Box domain.

OP-TEE-VM1, VM2 and VM3 are completely isolated in physical memory and do not affect each other.

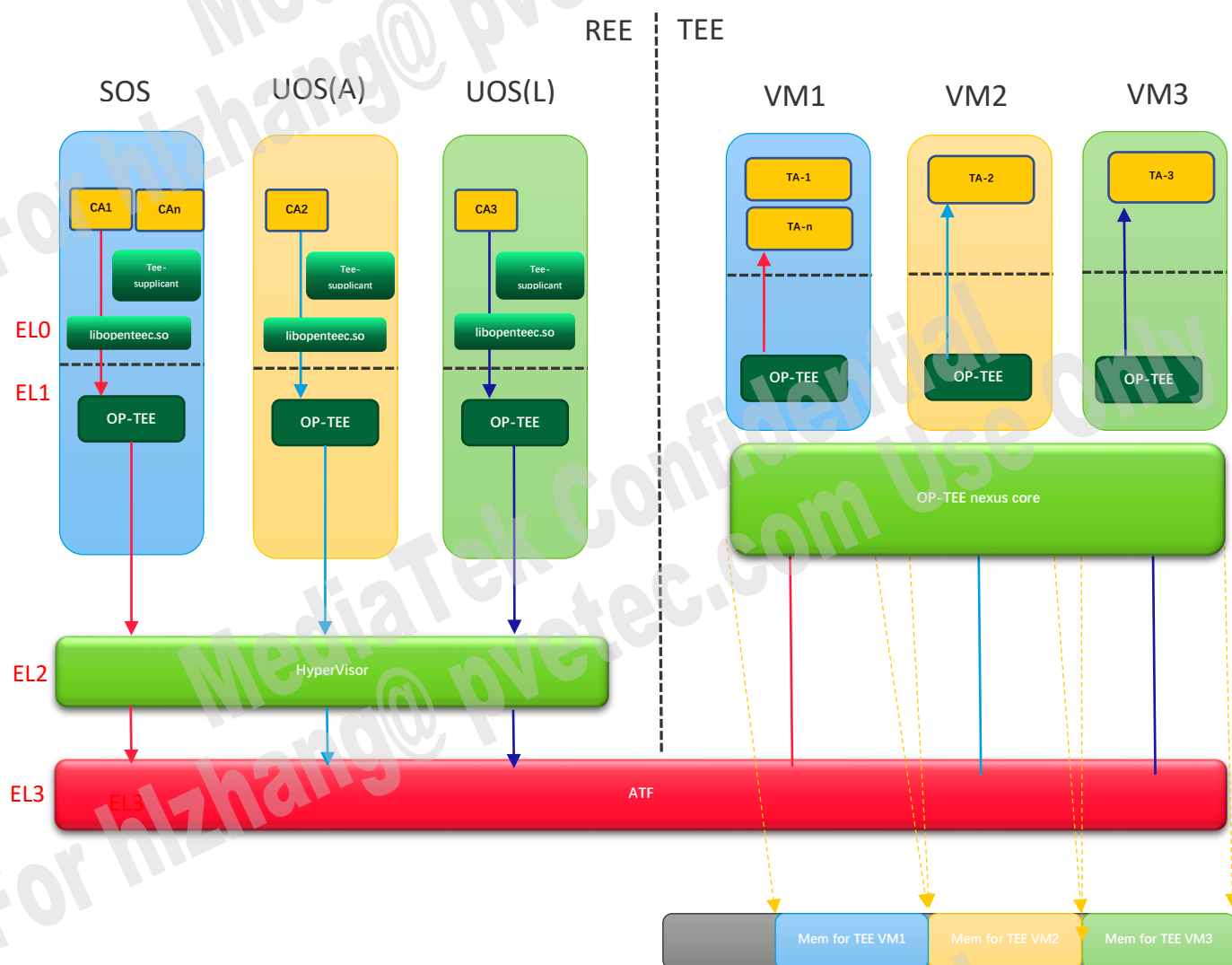


Figure 2-2. Composition of OP-TEE with virtualization

2.2 Enabling OP-TEE on MediaTek Platform

Modifications must be made to both the Yocto and Android branches to enable or disable OP-TEE.

The following sections describe the changes needed for the Yocto branch and the Android branch, respectively.

2.2.1 Yocto Branch

Check whether the following directories exist. If any of these directories are missing, this indicates an incomplete release, and you can contact MediaTek for assistance.

- `src/bsp/trustzone/optee/3.18.0/optee_client`
- `prebuilt/bsp/trustzone/optee/3.18.0/optee_os`
- `prebuilt/bsp/trustzone/optee/3.18.0/source`
- `prebuilt/bsp/trustzone/optee/3.18.0/ta/pkcs11`

After confirming that all directories exist, here is the method for configuring OP-TEE:

Yocto Branch Configuration

There are two configurations related to OP-TEE in the Yocto branch, each serving a specific function:

- **TEE_SUPPORT:** Indicates which TEE is supported. Setting it to “optee” enables OP-TEE.
- **TEE_TEST_SUPPORT:** Indicates whether to include the OP-TEE test tool xtest during compilation. This configuration is only effective for OP-TEE.

The configurations are defined in the Yocto conf file.

The reference path for the public version is: `meta/meta-mediatek-mt8676-hyp/conf/machine/auto8676p1_64_hyp.conf`

Please modify the corresponding location based on your project.

To enable OP-TEE, set the configurations as follows:

```
TEE_SUPPORT = "optee"
TEE_TEST_SUPPORT = "yes"
```

To disable OP-TEE, set the configurations as follows:

```
TEE_SUPPORT = "none"
TEE_TEST_SUPPORT = "no"
```

Kernel Configuration

The required modifications for the kernel are as follows:

- Enable the following two configurations in the kernel:

```
CONFIG_MTK_OPTEE_SUPPORT=m
CONFIG_OPTEE_REE_CONSOLE=y
```

In newer versions, setting TEE_SUPPORT to “optee” automatically enables these two kernel configurations. For older versions that do not adopt this mechanism, manually enable these configurations in the corresponding kernel configuration.

- Add the required kernel modules to the `.ko` table

- `tee.ko, ../../kernel_device_modules-6.1/drivers/tee/optee/tee/tee.ko, ramdisk, Y, Y, user/userdebug/eng`
- `optee.ko, ../../kernel_device_modules-6.1/drivers/tee/optee/tee/optee.ko, ramdisk, Y, Y, user/userdebug/eng`

c. Add the following description to the dts

```
&firmware {
    optee {
        compatible = "linaro,optee-tz";
        method = "smc";
    };
};
```

2.2.2 Android Branch

Please check if the following directories exist; if any of these directories are missing, it indicates an incomplete release, and you should contact MediaTek for assistance:

- vendor/mediatek/proprietary/trustzone/optee/3.18.0/optee_client
- vendor/mediatek/proprietary/trustzone/optee/3.18.0/secure_spmlib

Android Branch Configuration

There are two configurations related to OP-TEE in the Android branch, each serving a specific function:

- **MTK_OPTEE_SUPPORT:** Indicates whether OP-TEE is enabled at the vendor layer. If enabled, the OP-TEE binaries will be packaged into tee.img.
- **MGVI_MTK_OPTEE_SUPPORT:** An HAL layer configuration. This configuration does not affect the enabling or disabling of OP-TEE, but Android domain features based on OP-TEE might use this configuration to determine their own enabling or disabling. Therefore, it is recommended to enable this as well.

The configurations are defined in the following locations:

The definition of MTK_OPTEE_SUPPORT occurs in ProjectConfig.mk.

Example file path for the MediaTek public version:

- device/mediateksample/auto8676p1_64_bsp_vm/ProjectConfig.mk

MGVI_MTK_OPTEE_SUPPORT is defined in VendorConfig.mk

Example file path for the MediaTek public version:

- device/mediatek/vendor/mgvi_spm_64_armv82_tbox/VendorConfig.mk

To enable OP-TEE, set the configurations as follows:

```
MTK_OPTEE_SUPPORT = yes
MGVI_MTK_OPTEE_SUPPORT = yes
```

To disable OP-TEE, set the configurations to 'no' or simply remove the definitions:

The required modifications for the kernel are identical to those for Yocto. Refer to the previous section for details. Use the same method to enable OP-TEE configurations in the Android domain kernel layer.

2.3 Secure Storage Mechanism

Secure storage in OP-TEE is implemented according to the GlobalPlatform TEE core API definition. This specification requires storing general data and key materials. It ensures confidentiality and integrity of stored data, as well as atomicity of storage modification operations. Atomicity means that the entire operation either completes successfully or no write occurs. Currently, there are two secure storage implementations in OP-TEE:

2.3.1 REE File System

Uses an external (non-secure) file system to store encrypted data. This method relies on the file system of the Normal World, but all sensitive operations (such as encryption and decryption) are performed within the Secure World. The implementation principle is illustrated in Figure 2-3.

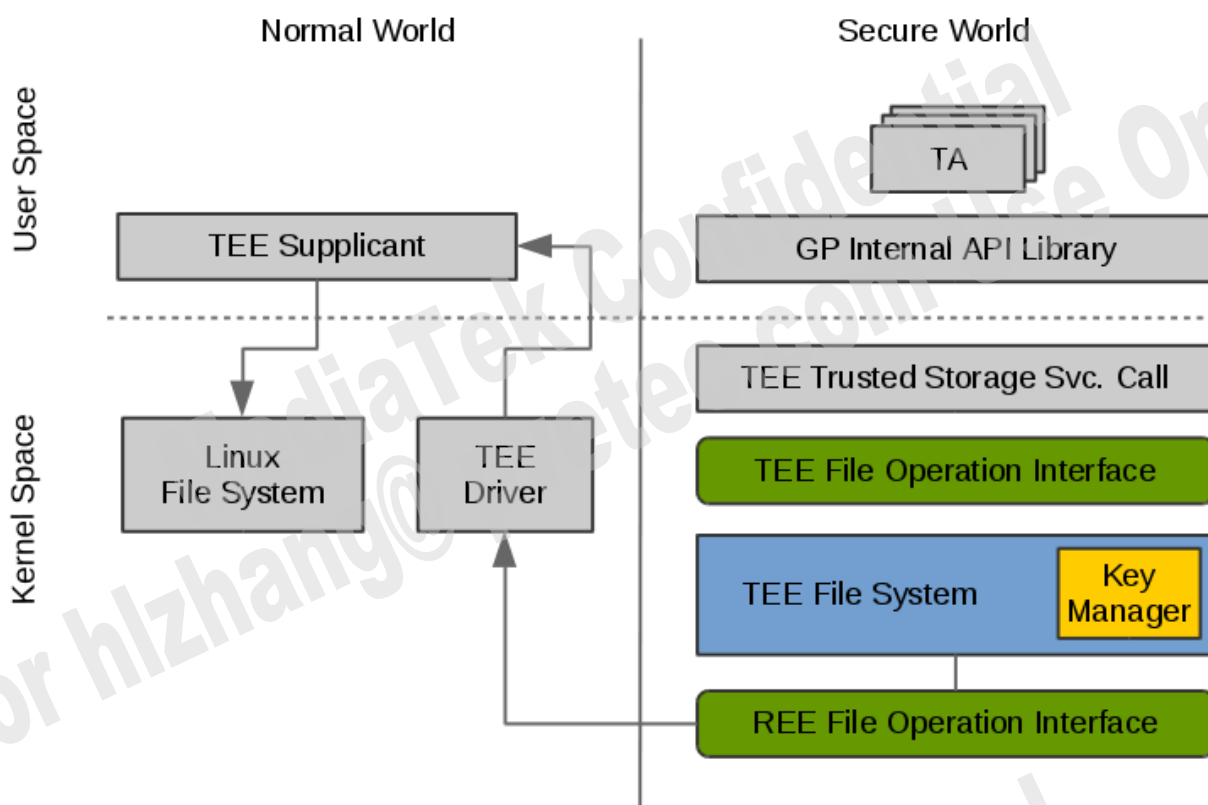


Figure 2-3. Secure storage based on OP-TEE

This type of secure storage is supported by OP-TEE by default.

Note:

- Data saved using the REE FS form of secure storage will be stored in the file system of the corresponding VM. Users or programs cannot read the contents of the file from the REE side, but users or programs with read/write permissions may tamper with or delete this file.

The default storage location in the Yocto domain is: `/var/lib/tee` or `/data/vendor/tee`

To modify this path, follow the steps below:

In the Yocto branch, locate:

```
meta/meta-mediatek/recipes-bsp/trustzone/optee-client_3.18.0.bb
```

Modify the definition of CFG_TEE_FS_PARENT_PATH.

The default storage location in the Android domain is: /data/vendor/tee

In the Android branch, locate:

```
vendor/mediatek/proprietary/trustzone/optee/3.18.0/optee_client/Android.bp
```

Modify the definition of -DTEE_FS_PARENT_PATH.

Note:

- Please modify this directory according to your needs. When making changes, ensure that the tee-supplciant has permission to create the directory and that the partition where the directory is located will not be reset due to a reboot or factory reset.

2.3.2 RPMB File System

For secure storage based on RPMB, data encryption and decryption methods resemble those utilized in the REE File System (FS) method. However, the storage medium changes from a regular file system to the RPMB area attached to UFS or Embedded MultiMediaCard (eMMC). Utilizing tamper-proof and replay-protection features of RPMB enhances security. The storage principle aligns with that depicted in [Figure 2-3](#), except that the "Linux File System" in the lower left corner is replaced with the RPMB driver.

To support RPMB-based secure storage in a virtualized project, the UFS hardware must support multiple RPMB regions (e.g., UFS 3.0 and above; eMMC does not support this). Therefore, this storage method is not supported by default, and you need to contact MediaTek to confirm whether this method is supported on your hardware.

When writing TA programs, you need to specify parameters to implement data storage using either the REE file system or the RPMB file system. Two OP-TEE specific storage identifiers are defined: TEE_STORAGE_PRIVATE_REE and TEE_STORAGE_PRIVATE_RPMB.

TEE_STORAGE_PRIVATE_REE indicates that data will be saved in the REE file system.

TEE_STORAGE_PRIVATE_RPMB indicates that data will be saved in the RPMB file system.

Note:

- Regardless of whether the REE FS or RPMB storage method is used, the data stored by each VM is not shared; that is, data written by one domain can only be read by that same domain.

Verify the functionality of secure storage on the platform by executing the xtest 6001 test item.

```
* regression_6001 Test TEE_CreatePersistentObject
o regression_6001.1 Storage id: 00000001
  regression_6001.1 OK
o regression_6001.2 Storage id: 80000000 -> REE FS secure storage
  regression_6001.2 OK
o regression_6001.3 Storage id: 80000100 -> RPMB secure storage
  regression_6001.3 OK
```

regression_6001 OK

If the regression_6001.3 test item is not executed during the execution of the xtest 6001 test, this indicates that the current device does not support RPMB. Possible reasons include unprovisioned RPMB keys or an inadequate software and hardware environment. For RPMB support, contact MediaTek for further investigation.

Refer to [Section 4.1](#) of this document for information on executing xtest.

2.4 Introduction to CA and TA

In OP-TEE, CA typically refers to “Client Application”. A CA represents an application operating in the Normal World (REE) that communicates with a Trusted Application (TA) functioning in the Trusted World (TEE). The TA is requested by the CA to perform secure operations through invocation of the TEE API.

The basic interaction flow between CA and TA is as follows:

1. CA Launch: The client application is launched in the Normal World.
2. Initialize TEE Session: The CA initializes a session with the TA through the TEE API.
3. Send Command: The CA sends a command request to the TA, asking it to perform specific secure operations.
4. TA Processes Request: The TA processes the request in the Trusted World and returns the results.
5. Receive Result: The CA receives the result returned by the TA and processes it as needed.
6. Close Session: After the operation is complete, the CA closes the session with the TA.

CAs can be written in either user space or kernel space based on specific needs.

In OP-TEE, TA refers to “Trusted Application”. OP-TEE supports three types of Trusted Applications (TAs): User Mode Trusted Applications (UTA), Static Trusted Applications (PTA, also known as Pseudo TA), and Early User Mode Trusted Applications (Early UTA). These application types differ in terms of functionality, execution timing, and runtime environment. The following are their main differences:

User Mode Trusted Application (UTA)

- Runtime Environment: UTAs run in user mode, as opposed to kernel mode. This means they execute in an isolated environment, providing higher security.
- Deployment: UTAs are typically deployed as separate applications and can be loaded and managed through secure storage.
- Use Case: UTAs are suitable for handling highly sensitive data or performing security-critical tasks, such as encryption operations and key management.

Static Trusted Application (PTA)

- Runtime Environment: PTAs run in the TEE kernel mode. Compared to UTAs, they have higher privileges and more direct access to TEE core services.
- Deployment: PTAs are usually statically integrated during the TEE OS build process and are not loaded as separate applications.
- Use Case: PTAs are used to implement functions that require higher privileged access to TEE resources, such as managing and controlling other components or services within the TEE.

Early User Mode Trusted Application (UTA)

- Runtime Environment: Early UTAs also run in user mode, but are loaded and executed at an early stage of system startup.
- Deployment: Similar to regular UTAs, but they start earlier during the TEE initialization process to handle early-stage security tasks.
- Use Case: Early UTAs are typically used to perform security checks or configurations during the system startup process, such as verifying the integrity of the boot configuration or securely configuring early hardware resources.

Note:

- Currently, OP-TEE based on the MediaTek platform only supports the development of UTAs. If you have development needs for PTAs or early UTAs, please contact MediaTek.

UTAs are placed in the file system:

- Yocto domain path: `/lib/optee_armtz/`
- Android domain path: `/vendor/lib/optee_armtz/`

If a TA is required in both the Yocto and Android domains, a copy must be placed in the file systems of both domains.

3 Application Development Based on OP-TEE

Both the Yocto and Android domains support OP-TEE, and the APIs required for the Yocto and Android domains are the same. This means that the CA and TA code for the Yocto and Android domains can be exactly the same. If your program needs to run in the Yocto domain, you need to package the CA and TA executable files into the Yocto domain's file system. Similarly, if it needs to run in the Android domain, you need to package the executable files into the Android domain's file system.

3.1 Writing CA and TA with GP API

OP-TEE supports the GP (GlobalPlatform) API, which means that your CA and TA programs can be easily ported to other TEE OSes that support the GP API. You can find information about the GP API on the internet.

The GP API used on the CA side starts with TEEC. You can find the relevant definitions in the header file located at `${PATH}/optee/3.18.0/optee_client/public/tee_client_api_extensions.h`.

- In the Yocto branch, the path is: `src/bsp/trustzone`
- In the Android branch, the path is: `vendor/mediatek/proprietary/trustzone`

The main APIs responsible for communication with the TEE on the CA side are as follows:

1. `TEEC_Result TEEC_InitializeContext(
 const char* name,
 TEEC_Context* context)`
2. `void TEEC_FinalizeContext(
 TEEC_Context* context)`
3. `TEEC_Result TEEC_OpenSession (
 TEEC_Context* context,
 TEEC_Session* session,
 const TEEC_UUID* destination,
 uint32_t connectionMethod,
 const void* connectionData,
 TEEC_Operation* operation,
 uint32_t* returnOrigin)`
4. `void TEEC_CloseSession (
 TEEC_Session* session)`
5. `TEEC_Result TEEC_InvokeCommand(
 TEEC_Session* session,
 uint32_t commandID,
 TEEC_Operation* operation,
 uint32_t* returnOrigin)`

The APIs related to shared memory operations on the CA side are as follows:

1. `TEEC_Result TEEC_RegisterSharedMemory(TEEC_Context *context, TEEC_SharedMemory *sharedMem);`
2. `TEEC_Result TEEC_AllocateSharedMemory(TEEC_Context *context, TEEC_SharedMemory *sharedMem);`
3. `void TEEC_ReleaseSharedMemory(TEEC_SharedMemory *sharedMemory);`

The GP APIs within the TEE start with `TEE_` and involve various operations within the TEE, such as encryption and decryption operations, secure file read and write operations, etc. For detailed information, please refer to the relevant GP documentation, which will not be elaborated here.

To write a TA, you must implement the following five APIs:

1. `TEE_Result TA_CreateEntryPoint(void)`
2. `void TA_DestroyEntryPoint(void)`
3. `_Result TA_OpenSessionEntryPoint(uint32_t param_types, TEE_Param __maybe_unused params[4], void __maybe_unused **sess_ctx)`
4. `void TA_CloseSessionEntryPoint(void __maybe_unused *sess_ctx)`
5. `TEE_Result TA_InvokeCommandEntryPoint(void __maybe_unused *sess_ctx, uint32_t cmd_id, uint32_t param_types, TEE_Param params[4])`

Next, the process of writing a simple CA and TA using an example program will be explained.

3.2 Example Program

A sample code has been written by MediaTek to demonstrate how to create a simple demo program. This sample code is applicable in both the Yocto and Android domains. You can contact MediaTek to obtain the complete source files. The demo code provided by MeidaTek will be explained in this chapter.

3.2.1 CA Files

The development of the CA part mainly involves using the TEE Client API introduced in [Section 3.1](#), including how to initialize the context, open/close sessions, and use different parameter types to interact with the TA. You can refer to the demo's call flow for CA development.

The following is the process for initializing and destroying the TEE context as well as opening and closing sessions. If there is no need for multi-instance TA concurrency, it is recommended to refer to the demo and use global variables to control the context and session count to a single instance.

```
static TEEC_Session g_stTeecSession = {0};
static TEEC_Context g_stTeecContext = {0};
static bool g_bIsTeecInitialized = false;

TEEC_Result initTzContext()
{
    const TEEC_UUID stTeecUuid = {0xa60589fd, 0x0158, 0x40eb, {0xa7, 0x74, 0xa3,
0x98, 0xb1, 0xd6, 0x98, 0x32}};
    TEEC_Result u32TeecResult = TEEC_ERROR_GENERIC;
    uint32_t u32RetOrig = 0;

    if (true == g_bIsTeecInitialized)
    {
        u32TeecResult = TEEC_SUCCESS;
        goto exit;
    }

    u32TeecResult = TEEC_InitializeContext(NULL, &g_stTeecContext);
    if (TEEC_SUCCESS != u32TeecResult)
    {
        DLOGE("TEEC_InitializeContext fail, %X", u32TeecResult);
        goto exit;
    }

    u32TeecResult = TEEC_OpenSession(&g_stTeecContext, &g_stTeecSession,
&stTeecUuid, TEEC_LOGIN_PUBLIC, NULL, NULL, &u32RetOrig);
    if (TEEC_SUCCESS != u32TeecResult)
    {
        TEEC_FinalizeContext(&g_stTeecContext);
        DLOGE("TEEC_OpenSession fail, %X, orig: %X", u32TeecResult, u32RetOrig);
        goto exit;
    }

    g_bIsTeecInitialized = true;
    exit:
    return u32TeecResult;
}

TEEC_Result uninitTzContext()
{
    TEEC_Result u32TeecResult = TEEC_ERROR_GENERIC;

    if (false == g_bIsTeecInitialized)
    {
        u32TeecResult = TEEC_ERROR_GENERIC;
        DLOGE("Session not exist!");
        goto exit;
    }
}
```

```

TEEC_CloseSession(&g_stTeecSession);
TEEC_FinalizeContext(&g_stTeecContext);

u32TeecResult = TEEC_SUCCESS;
g_bIsTeecInitialized = false;

exit:
return u32TeecResult;
}

```

When the CA sends a command to the TA, it needs to use the TEEC_InvokeCommand function. This function supports passing different types of parameters from the CA to the TA. The following is an example of how to use this function with value-type parameters.

```

TEEC_Result testValueParameter()
{
    TEEC_Result u32TeecResult = TEEC_ERROR_GENERIC;
    uint32_t u32RetOrig = 0;
    TEEC_Operation stTeecOperation = {0};

    memset(&stTeecOperation, 0, sizeof(TEEC_Operation));
    stTeecOperation.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INPUT,
        TEEC_NONE,
        TEEC_NONE,
        TEEC_NONE);
    stTeecOperation.params[0].value.a = 0x11223344;
    stTeecOperation.params[0].value.b = 0x55667788;
    u32TeecResult = TEEC_InvokeCommand(&g_stTeecSession, E_DEMO_CMD_VALUE_INPUT_TEST,
        &stTeecOperation, &u32RetOrig);
    if ((TEEC_SUCCESS != u32TeecResult) || (TEEC_ORIGIN_TRUSTED_APP != u32RetOrig))
    {
        DLOGE("TEEC_InvokeCommand fail, result=%X, origin=%X", u32TeecResult, u32RetOrig);
        goto exit;
    }

    exit:
    return u32TeecResult;
}

```

3.2.2 TA Files

The TA part implements various operations requested by the CA based on different command identifiers. The TA developer needs to define their own UUID, which is passed in when the CA calls TEEC_OpenSession. Additionally, this demo defines the TA as a single instance in user_ta_header_defines.h, meaning that the TA binary will only be loaded once in OP-TEE memory and will not be loaded multiple times. The TA_STACK_SIZE and TA_DATA_SIZE need to be adjusted according to the implementation of each TA.

```

#define TA_UUID { 0xa60589fd, 0x0158, 0x40eb, {0xa7, 0x74, 0xa3, 0x98, 0xb1, 0xd6,
0x98, 0x32} }

```

```
TEE_Result TA_CreateEntryPoint(void)
```

```
#define TA_FLAGS (TA_FLAG_SINGLE_INSTANCE | \
TA_FLAG_MULTI_SESSION \
)
```

```
#define TA_STACK_SIZE (2 * 1024)
```

```
#define TA_DATA_SIZE (32 * 1024)
```

Upon receipt of a command sent by the CA, the parameters are checked based on the command identifier, and the corresponding functionality is implemented.

```
TEE_Result TA_InvokeCommandEntryPoint(void *pSessionContext, uint32_t nCommandID,
uint32_t nParamTypes, TEE_Param pParams[4])
```

```
{
    (void)pSessionContext;
    DLOGE("[TA]CommandID :%d \n", nCommandID);
    switch (nCommandID)
    {
        case E_DEMO_CMD_NO_PARA_TEST:
            (void)pParams;
            return demo_ta_no_para_test(nParamTypes);
        case E_DEMO_CMD_VALUE_INPUT_TEST:
            return demo_ta_value_input_test(nParamTypes,pParams);
        case E_DEMO_CMD_VALUE_OUTPUT_TEST:
            return demo_ta_value_output_test(nParamTypes,pParams);
        case E_DEMO_CMD_VALUE_INOUT_TEST:
            return demo_ta_value_inout_test(nParamTypes,pParams);
        case E_DEMO_CMD_MEMREF_TEMP_INPUT_TEST:
            return demo_ta_memref_temp_input_test(nParamTypes,pParams);
        case E_DEMO_CMD_MEMREF_TEMP_OUTPUT_TEST:
            return demo_ta_memref_temp_output_test(nParamTypes,pParams);
        case E_DEMO_CMD_MEMREF_TEMP_INOUT_TEST:
            return demo_ta_memref_temp_inout_test(nParamTypes,pParams);
        default:
            DLOGE("Unexpected command id: %d", nCommandID);
            return TEE_ERROR_BAD_PARAMETERS;
    }
}
```

3.3 Compilation and Execution

The compilation of the CA requires the library file libopentec.so; the compilation of the TA requires the development package provided by MediaTek.

The development package for the Android domain is located at vendor/mediatek/proprietary/trustzone/optee/3.18.0/secure_spmlib.

The development package for the Yocto domain is located at prebuilt/bsp/trustzone/optee/3.18.0/optee_os.

3.3.1 Yocto Domain

For compiling CA/TA in the Yocto domain, it is recommended to use the native Yocto method. You can refer to the following example:

```
inherit fog
inherit deploy

DESCRIPTION = "demo_tz_app"
LICENSE = "MediaTekProprietary"
LIC_FILES_CHKSUM =
"file://${PROPRIETARY_LICENSE_PATH}/MediaTekProprietary;md5=c5d17c6905715d0948a3d60876
02d12d"

SRC_URI = "fog://src/bsp/tzapp2/demo_tz_app;name=demo_tz_app"
PACKAGES = "${PN}"
INHIBIT_PACKAGE_DEBUG_SPLIT = "1"

SRCREV_demo_tz_app = "${AUTOREV}"
S = "${WORKDIR}/git"

DEPENDS += "optee-client"
DEPENDS += "optee-os-tadevkit"

BUILD_FROM_YOCTO ?= "true"
TA_DEV_KIT_DIR = "${STAGING_INCDIR}/optee/export-user_ta"

EXTRA_OEMAKE = " \
CROSS_COMPILE=${TARGET_PREFIX} \
TA_DEV_KIT_DIR=${TA_DEV_KIT_DIR} \
BUILD_FROM_YOCTO=${BUILD_FROM_YOCTO} \
"

EXTRA_OEMAKE:append = "LIBGCC_LOCATE_CFLAGS=--sysroot=${STAGING_DIR_HOST}"

INSANE_SKIP:${PN} += "already-stripped"
FILES:${PN} += "${libdir}"
FILES:${PN} += "${bindir}"
FILES:${PN} += "${nonarch_base_libdir}/optee_armtz/"

do_compile() {
if [ -e makefile ]; then
oe_runmake
fi
}

do_install () {
if [ -e makefile ]; then
oe_runmake O="${WORKDIR}" DESTDIR="${D}" install
fi
}
```

In `src/bsp/tzapp2/demo_tz_app`, a makefile is provided for the bb file to call.

```
.PHONY: all clean install

all:
$(MAKE) -C demoCA setup
$(MAKE) -C demoCA build BUILD_FROM_YOCTO=${BUILD_FROM_YOCTO}
$(MAKE) -C demoTA TA_DEV_KIT_DIR=${TA_DEV_KIT_DIR}
LIBGCC_LOCATE_CFLAGS=${LIBGCC_LOCATE_CFLAGS}

clean:
$(MAKE) -C demoCA clean
$(MAKE) -C demoTA clean

install:
install -d $(DESTDIR)/usr/bin
install -m 755 demoCA/out/demoTzApp $(DESTDIR)/usr/bin
install -d ${DESTDIR}${nonarch_base_libdir}/optee_armtz
install -m 0444 demoTA/out/*.ta ${DESTDIR}${nonarch_base_libdir}/optee_armtz

uninstall:
rm -f $(DESTDIR)/usr/bin/demoTzApp
rm -f ${DESTDIR}${nonarch_base_libdir}/optee_armtz/$(BINARY).ta
```

The makefile for the CA is as follows:

```
TOP_DIR = ${CURDIR}
OBJ_PATH = ./out
TARGET := demoTzApp

SRC += $(TOP_DIR)/src/main.c
INC += -I./include
LIBS += -lopenteec

CFLAGS := $(INC) \
-Werror \
-fPIC -g -O2 \
-fstack-protector
ifeq ($(BUILD_FROM_YOCTO), true)
CFLAGS += -DANDROID_BUILD=0
endif

OBJS := $(SRC:.c=.o)
OBJS_S := $(SRC_S:.S=.o)

.PHONY: all

all: clean setup build

setup:
@mkdir -p $(OBJ_PATH)

build: $(OBJS) $(OBJS_S)
$(CC) $(CFLAGS) $(LDFLAGS) -o $(OBJ_PATH)/$(TARGET) $(OBJ_PATH)/*.o $(LIBS)
```

```
$(OBJS): %.o:%.c
$(CC) $(CPPFLAGS) $(CFLAGS) -o $(OBJ_PATH)/$(notdir $@) -c $<

$(OBJS_S): %.o:%.S
$(CC) $(CPPFLAGS) $(CFLAGS) -o $(OBJ_PATH)/$(notdir $@) -c $<

clean:
    @rm -rf $(OBJ_PATH)
```

The compilation of the TA requires both a makefile and a sub.mk file.

The makefile is as follows:

```
ifneq ($O,)
out-dir := $O
else
out-dir := $(CURDIR)/out
endif
O=$(out-dir)

BINARY = a60589fd-0158-40eb-a774-a398b1d69832
export BINARY
override LDADD += -L$(O)/../

-include $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk

ifeq ($(wildcard $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk), )
clean:
@echo 'Note: $$$(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk not found, cannot clean TA'
@echo 'Note: TA_DEV_KIT_DIR=$(TA_DEV_KIT_DIR)'
endif
```

sub.mk is as follows:

```
global-incdirs-y += include
srcs-y += src/demo_ta_entry.c
srcs-y += src/demo_ta_commands.c
```

The compilation command reference is as follows:

```
source meta/poky/oe-init-build-env && bitbake demo-tz-app -c cleansstate; bitbake demo-tz-app 2>&1 | tee mtk_demo.log
```

The command compiles both the CA and TA files.

You can view the compiled artifacts at the following paths:

- **Yocto CA:**
build/tmp/work/aarch64-poky-linux/demo-tz-app/1.0/git/demoCA/out/demoTzApp
- **Yocto TA:**
build/tmp/work/aarch64-poky-linux/demo-tz-app/1.0/git/demoTA/out/a60589fd-0158-40eb-a774-a398b1d69832.ta

Push the TA (a60589fd-0158-40eb-a774-a398b1d69832.ta) into the `/lib/optee_armtz/` directory on the platform; Push the CA (demoTzApp) into any writable directory on the platform, change its permissions to executable, and then you can run the demo program.

3.3.2 Android Domain

For the Android domain, it is recommended to use the built-in Android build system for compiling the CA. The `Android.bp` file can be written as follows:

```
cc_binary {
  name: "demoTzApp",
  vendor: true,
  owner: "mtk",
  proprietary: true,
  cflags: ["-DANDROID_BUILD=1"],
  local_include_dirs: ["include"],
  srcs: ["src/main.c"],
  shared_libs: ["liblog", "libopentee"],
  compile_multilib: "both",
}
```

Using MT8676 platform as an example, the compilation command is:

```
source build/envsetup.sh && export OUT_DIR=out_demo && lunch vext_auto8676p1_64_bsp_vm_tbox-
userdebug && mmm vendor/mediatek/proprietary/trustzone/demo_tz_app/demoCA | tee
make_demoCA.log
```

For the Android domain, TA compilation uses a script. Refer to the following example:

```
export TA_DEV_KIT_DIR=$PWD/export-ta_arm64
if [ -d ${TA_DEV_KIT_DIR} ]; then
echo "TA_DEV_KIT_DIR = ${TA_DEV_KIT_DIR}"
else
tar xzf tz_optee_3.18.0_dev_kit.tar.gz
echo -e "\033[1;32m #### uncompress: TA_DEV_KIT_DIR = ${TA_DEV_KIT_DIR}. ####\033[m"
fi

#### !!! Need to modify ANDROID_TOP !!! ###
ANDROID_TOP=/xxx/alps-mp-xxx-20xx_xx_xx_xx_xx
export PATH=${ANDROID_TOP}/prebuilts/clang/clang-tee/linux-x86/bin/:${PATH}
export COMPILER=clang
export O=out

make clean
make
```

Place the Android OP-TEE TA development package provided by MediaTek, along with the TA compilation makefile and `sub.mk` from [Section 3.3.1](#), in the same directory for compilation.

Note:

- Before using this script, you need to set the `ANDROID_TOP` value to the path of the MediaTek Android baseline.

You can view the compiled artifacts at the following path:

- **Android CA:**
`out_demo/target/product/auto8676p1_64_bsp_vm_tbox/vendor/bin/demoTzApp`
- **Android TA:**
`vendor/mediatek/proprietary/trustzone/demo_tz_app/demoTA/out/a60589fd-0158-40eb-a774-a398b1d69832.ta`

Push the TA (a60589fd-0158-40eb-a774-a398b1d69832.ta) into the `/vendor/lib/optee_armtz/` directory on the platform; Push the CA (demoTzApp) into any writable directory on the platform, change its permissions to executable, and then you can run the demo program.

4 OP-TEE Testing and Debugging

4.1 OP-TEE Self-Test Method

The official self-test tool of OP-TEE is called xtest.

In the Yocto domain, the xtest code will be released by MediaTek and can be compiled and packaged into the load.

In the Android domain, due to release policy reasons, the xtest source code is not provided by MediaTek. Push the binary files to the platform for testing.

You can review the source code to observe the test items. Table 4-1 shows a brief summary of the content:

Table 4-1. Xtest test items

Xtest Item	Test Purpose Description
xtest_1	Basic functionality tests of OP-TEE, such as entering and exiting the TEE, TA panic tests, etc.
xtest_2	Socket-related tests
xtest_4	Encryption and decryption algorithm test
xtest_5	Shared memory tests
xtest_6	Secure storage-related tests
xtest_8	Miscellaneous tests such as KDF, Mbed, etc.
Pkcs11	Tests related to PKCS #11 components

Within the Yocto domain

xtest consists of the following files:

/usr/bin/xtest -> The executable program for xtest

/lib/optee_armtz/*.ta -> UTAs that xtest depends on

Start executing xtest by entering xtest in the serial console or adb shell.

In the Android domain, xtest consists of the following files:

/vendor/bin/xtest -> The executable program for xtest

/vendor/lib/optee_armtz/*.ta -> UTAs that xtest depends on

Before executing xtest in the Android domain, you need to switch to administrator privileges using su, disable SELinux with setenforce 0, and then execute xtest.

```
# su
# setenforce 0
# xtest
```

Upon completion of xtest, the number of test items that passed and failed will be printed.

If xtest cannot be executed at all, it indicates that the current OP-TEE environment is abnormal, possibly due to an incomplete OP-TEE setup. Please perform the following self-checks:

1. Check if optee.ko and tee.ko are correctly inserted:

```
# lsmod | grep tee
```

Under normal circumstances, the TEE and OP-TEE drivers should be correctly loaded.

2. Check if the tee-supplciant process is running: Enter the folloiwng command:

```
# ps -A | grep tee
```

Verify the existence of the tee-supplciant process.

Under normal circumstances, you should see the tee-supplciant process running in the background; for example:

```
adb -e shellsh-3.2# ps -A | grep tee
  2250 ?          00:00:00 tee-supplciant

auto8676p1_64_bsp_vm_tbox:/ $ ps -A | grep tee
nobody      451      1      2452248  13708 0          0 S
android.hardware.security.keymint@3.0-service.tee
system      803      1      2191704  10548 0          0 S
android.hardware.gatekeeper-service.tee
root        1125      1      2176420  10168 0          0 S tee-supplciant
```

4.2 Introduction to OP-TEE Related Logs

As introduced earlier, the function call flow from CA to TA generally involves the following parts:

CA -> libopentec.so -> Linux kernel -> ATF -> OP-TEE OS -> TA

How the logs are printed in each of these modules will be explained when introducing logs.

The logic in libopentec.so, the Linux Kernel, and ATF remains relatively simple and does not typically encounter issues.

The most common problems generally occur in the CA, TA, and OP-TEE OS.

4.2.1 User Space

User space includes the CA and libopentec.so.

CA Logs: The logs on the CA side are written and maintained by the application developer. For example, if it is a Yocto domain application, you can use printf to print logs. If it is an Android domain application, logs are output using Android-specific logging methods such as ALOG.

Note:

- The last parameter `err_origin` of the `TEEC_OpenSession` and `TEEC_InvokeCommand` APIs can indicate the stage at which an error occurs. It is strongly recommended that application developers print this log.

For example:

```
res = TEEC_OpenSession(&ctx, &sess, &uuid,
TEEC_LOGIN_PUBLIC, NULL, NULL, &err_origin);
if (res != TEEC_SUCCESS)
errx(1, "TEEC_Opensession failed with code 0x%x origin 0x%x", res, err_origin);

res = TEEC_InvokeCommand(&sess, TA_HELLO_WORLD_CMD_INC_VALUE, &op, &err_origin);
if (res != TEEC_SUCCESS)
errx(1, "TEEC_InvokeCommand failed with code 0x%x origin 0x%x", res, err_origin);
```

The err_origin values range from 1 to 4, with the following meanings:

- TEEC_ORIGIN_API (0x00000001): An error occurring at the user space layer.
- TEEC_ORIGIN_COMMS (0x00000002): An error occurring in Linux/ATF communication.
- TEEC_ORIGIN_TEE (0x00000003): An error occurring at the OP-TEE Kernel layer.
- TEEC_ORIGIN_TRUSTED_APP (0x00000004): Error occurring at the TA layer.

libopenteec.so links with the CA code written by the developer. The log print method is the same as that of CA.

4.2.2 Linux Kernel

Error logs in the OP-TEE Linux Kernel are generated by pr_err and displayed in dmesg.

4.2.3 Secure World

The Secure World comprises both the OP-TEE OS and TA.

OP-TEE OS: Defines four log levels: FMSG, DMSG, IMMSG, and EMSG.

The default log level for the OP-TEE OS is 1. This configuration allows only EMSG logs to be output.

For security reasons, the OP-TEE log level cannot be dynamically adjusted. If you need to adjust the log level, you must modify the OP-TEE configuration and recompile OP-TEE. The modification path is:

vendor/mediatek/proprietary/trustzone/optee/3.18.0/optee_os/mk/config.mk

```
# Log levels for the TEE core. Defines which core messages are displayed
# on the secure console. Disabling core log (level set to 0) also disables
# logs from the TAs.
# 0: none
# 1: error
# 2: error + info
# 3: error + info + debug
# 4: error + info + debug + flow
CFG_TEE_CORE_LOG_LEVEL ?= 1
```

Note:

- MediaTek releases the OP-TEE image in binary form, with the log level set to 1 by default. If you need to adjust the log level of the TEE OS, please contact MediaTek.

The OP-TEE OS layer has added a Guest ID to indicate which VM the current code is executing in; for example:

E/TC:1 5 0 xxxxx

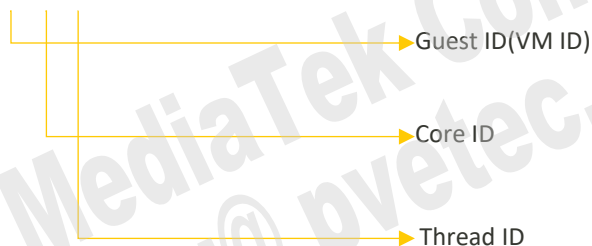


Figure 4-1. OP-TEE log

The TA logs are designed similarly to the OP-TEE OS logs, with four log levels defined: FMSG, DMSG, IMMSG, and EMSG. The default log level is 1, which only allows EMSG logs to be output. If you wish to adjust the log level of the TA, the TA developer can freely do so. The log level of the TA can be set by defining the value of CFG_TEE_TA_LOG_LEVEL in the TA's makefile, with the default value being 1.

Note:

- Logs from S-EL0 (TA) and S-EL1 (OP-TEE) in the Secure World are sent back to the corresponding domain's kernel layer and output at the pr_info level. You can capture the kernel logs, which will include OP-TEE logs from S-EL0/1, using tools like dmesg. For example:

```
adb shell dmesg -w > log.txt
```

4.3 Common OP-TEE Exceptions and Analysis

The open-source community has documentation on how to analyze and handle OP-TEE abort dumps and call stacks. Please refer to: [OPTee Debug: Abort Dumps] (https://optee.readthedocs.io/en/latest/debug/abort_dumps.html)

A brief description is provided in this section based on the content of the link.

Upon encountering an abort or panic, panic logs are printed by the TEE OS or TA.

The log showing "TEE load address" and "call stack" indicates that an exception occurs in the OP-TEE OS.

```
E/TC:2 1 TEE load address @ 0x6cc30000
E/TC:2 1 Call stack:
E/TC:2 1 0x6ccb5fe0
```

The panic information of the OP-TEE OS will be analyzed by MediaTek.

When the exception occurs in the TA written by the developer, the developer needs to analyze the error cause themselves. Here, an example from the open-source community documentation is used to illustrate. When the call stack shows "Status of TA", it indicates that the exception occurs in the TA.

```
E/TC:0 TA panicked with code 0x0
E/TC:0 Status of TA 484d4143-2d53-4841-3120-4a6f636b6542 (0xe07ba50) (active)
E/TC:0 arch: arm load address: 0x101000 ctx-idr: 1
E/TC:0 stack: 0x100000 4096
```

```

E/TC:0 region 0: va 0x100000 pa 0xe31d000 size 0x1000 flags rw-
E/TC:0 region 1: va 0x101000 pa 0xe300000 size 0xf000 flags r-x
E/TC:0 region 2: va 0x110000 pa 0xe30f000 size 0x3000 flags r--
E/TC:0 region 3: va 0x113000 pa 0xe312000 size 0xb000 flags rw-
E/TC:0 region 4: va 0 pa 0 size 0 flags ---
E/TC:0 region 5: va 0 pa 0 size 0 flags ---
E/TC:0 region 6: va 0 pa 0 size 0 flags ---
E/TC:0 region 7: va 0 pa 0 size 0 flags ---
E/TC:0 Call stack:
E/TC:0 0x001044a8
E/TC:0 0x0010ba59
E/TC:0 0x00101093
E/TC:0 0x001013ed
E/TC:0 0x00101545
E/TC:0 0x0010441b
E/TC:0 0x00104477

```

The first analysis method is to use the script provided by the open-source community for direct parsing, which is recommended as the first choice:

```
$ cat dump.txt | ./optee_os/scripts/symbolize.py -d ./optee_examples/*/ta
```

If the first method does not resolve the issue, you can try manual parsing. The parsing method is as follows:

1. Find the corresponding `{uuid}.dmp` file for the TA.
 - When compiled in a Yocto environment, the dmp file is located under `build/tmp/work/xxxx`.
 - When compiled in an Android environment, the dmp file resides under `out`.
2. Calculate the offset by subtracting the TA's load address from the crash address. For example:
Offset calculation: $\text{Offset} = 0x001044A8 - 0x101000 = 34A8$
3. Use a text tool to open the `{uuid}.dmp` file.
4. Find the position corresponding to 34A8 in the `{uuid}.dmp` file to determine the approximate location of the panic.

5 Appendix

5.1 Reference Documents

1. OP-TEE Open Source Community Documentation:
<https://optee.readthedocs.io/en/latest/index.html>
2. Arm Developer Documentation on TrustZone:
<https://developer.arm.com/documentation/102418/0102?lang=en>

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.